Especificación de Requisitos de Software (SRS)

VotaGT - Sistema de Gestión Electoral

Norma de referencia: ISO/IEC/IEEE 29148:2018

Curso: Análisis de Sistemas II

Universidad: Universidad Mariano Gálvez de Guatemala

Versión: 1.0

Fecha: Octubre 2026

---

1. Introducción

1.1 Propósito

El presente documento define los requisitos funcionales y no funcionales del sistema VotaGT.

VotaGT es una plataforma para la gestión de procesos electorales que permite autenticar usuarios, administrar votaciones, candidatos y mesas, registrar resultados, consultar resultados electorales y generar reportes.

La especificación tiene como propósito establecer de forma clara, verificable y trazable el comportamiento esperado del sistema.

1.2 Alcance

VotaGT permite administrar el flujo principal de una votación mediante una API REST.

El alcance implementado comprende:

- Autenticación de usuarios.

- Autorización mediante roles y permisos.

- Gestión de votaciones.

- Gestión de candidatos.

- Gestión de mesas.

- Registro de resultados.

- Consulta de resultados.

- Exportación de resultados en JSON y CSV.

- Auditoría de operaciones.

- Caché de resultados.

- Representación de relaciones de autorización.

El backend está desarrollado utilizando Node.js y Express.

1.3 Definiciones y acrónimos

| Término | Definición |

|---|---|

| API | Application Programming Interface |

| REST | Representational State Transfer |

| JWT | JSON Web Token |

| SRS | Software Requirements Specification |

| RBAC | Control de acceso basado en roles |

| TTL | Time To Live |

| BDD | Behavior Driven Development |

| Cache-Aside | Patrón donde la aplicación consulta y actualiza explícitamente la caché |

| Problem Details | Formato estandarizado para representar errores HTTP |

| VotaGT | Sistema de Gestión Electoral desarrollado en el proyecto |

---

2. Descripción general

2.1 Perspectiva del producto

VotaGT se implementa como una aplicación backend basada en una arquitectura de Monolito Modular con API REST.

La solución utiliza persistencia políglota:

- PostgreSQL para información transaccional.

- MongoDB para auditoría.

- Redis para caché.

- Neo4j para relaciones de autorización.

2.2 Tipos de usuario

Administrador

Usuario con privilegios para realizar operaciones administrativas y electorales.

Digitador

Usuario encargado del registro de información electoral y resultados.

Supervisor

Usuario encargado de supervisar la información registrada y consultar resultados.

Usuario de consulta

Usuario con permisos principalmente de lectura.

2.3 Entorno de operación

El backend se ejecuta mediante:

- Node.js.

- Express.

- PostgreSQL 18.

- Redis 7.

- MongoDB 7.

- Neo4j 5 Community.

- Docker para servicios auxiliares.

La API local utiliza como dirección base:

```
http://localhost:3000/api/v1
```

---

3. Restricciones generales

El sistema deberá utilizar autenticación mediante JWT para los recursos protegidos.

El sistema deberá utilizar PostgreSQL como fuente principal de los datos electorales.

El sistema deberá utilizar permisos para controlar el acceso a operaciones protegidas.

El sistema deberá utilizar respuestas HTTP estandarizadas para representar operaciones exitosas y errores.

Los datos sensibles de configuración deberán obtenerse mediante variables de entorno.

---

4. Requisitos funcionales

REQ-FN-001 - Autenticación de usuario

Requisito:

El sistema **deberá validar las credenciales proporcionadas por un usuario y generar un token JWT cuando las credenciales sean válidas**.

Entrada

- Identificador de usuario.

- Contraseña.

Resultado esperado

El sistema devuelve HTTP 200 y un token JWT.

Excepciones

El sistema deberá rechazar credenciales incorrectas sin revelar si el usuario o la contraseña fue el dato inválido.

REQ-FN-002 - Consulta del perfil autenticado

Requisito:

El sistema **deberá proporcionar la información del usuario autenticado cuando la solicitud contenga un token JWT válido**.

Resultado esperado

La respuesta deberá incluir la información correspondiente al usuario y sus permisos.

Excepción

Cuando no exista un token válido, el sistema deberá responder HTTP 401.

REQ-FN-003 - Consulta de votaciones

Requisito:

El sistema **deberá permitir que un usuario autenticado consulte las votaciones registradas cuando posea acceso válido al sistema**.

Resultado esperado

HTTP 200 con las votaciones disponibles.

REQ-FN-004 - Creación de votaciones

Requisito:

El sistema **deberá permitir crear una votación cuando el usuario autenticado posea el permiso correspondiente y los datos proporcionados sean válidos**.

Condiciones

- El nombre deberá ser obligatorio.

- La fecha final deberá ser posterior a la fecha inicial.

Resultado esperado

HTTP 201.

REQ-FN-005 - Consulta individual de votación

Requisito:

El sistema **deberá permitir consultar una votación mediante su identificador cuando dicha votación exista**.

Excepción

Cuando la votación no exista, el sistema deberá responder HTTP 404.

REQ-FN-006 - Consulta de candidatos

Requisito:

El sistema **deberá mostrar los candidatos asociados a una votación cuando la votación solicitada exista**.

Excepción

Una votación inexistente deberá producir HTTP 404.

REQ-FN-007 - Registro de candidatos

Requisito:

El sistema **deberá permitir registrar un candidato en una votación cuando el usuario posea el permiso requerido y los datos sean válidos**.

Reglas

- Nombre obligatorio.

- Cargo obligatorio.

- El número de lista, cuando se utilice, deberá ser un entero positivo.

- El número de lista no deberá repetirse dentro de la misma votación.

Excepciones

Una duplicación deberá responder HTTP 409.

REQ-FN-008 - Consulta de mesas

Requisito:

El sistema **deberá mostrar las mesas pertenecientes a una votación cuando la votación exista**.

REQ-FN-009 - Registro de mesas

Requisito:

El sistema **deberá permitir registrar una mesa electoral cuando el usuario posea el permiso requerido y el código de la mesa no se encuentre registrado en la misma votación**.

Campos obligatorios

- Código.

- Nombre.

Excepción

Un código duplicado deberá responder HTTP 409.

REQ-FN-010 - Registro de resultados

Requisito:

El sistema **deberá permitir registrar el número de votos obtenido por un candidato en una mesa cuando la votación, la mesa y el candidato sean válidos**.

Reglas

- Los votos deberán ser un número entero.

- Los votos deberán ser mayores o iguales a cero.

- La mesa deberá pertenecer a la votación.

- El candidato deberá pertenecer a la votación.

- No deberá existir previamente un resultado para la misma combinación de mesa y candidato.

Resultado esperado

HTTP 201.

Excepciones

Los datos inválidos deberán responder HTTP 400.

Los recursos inexistentes deberán responder HTTP 404.

Un resultado duplicado deberá responder HTTP 409.

REQ-FN-011 - Consulta consolidada de resultados

Requisito:

El sistema **deberá proporcionar los resultados consolidados de una votación cuando la votación solicitada exista**.

La respuesta deberá permitir consultar:

- Identificador de la votación.

- Número de mesas con resultados.

- Total de votos registrados.

- Resultados por candidato.

El sistema podrá obtener la información desde Redis cuando exista una copia válida en caché o desde PostgreSQL cuando la caché no esté disponible.

REQ-FN-012 - Registro de auditoría

Requisito:

El sistema **deberá registrar eventos de auditoría cuando se ejecuten operaciones relevantes de seguridad o modificación de información**.

Entre los eventos se incluyen:

- Inicio de sesión exitoso.

- Creación de votación.

- Creación de candidato.

- Creación de mesa.

- Registro de resultado.

Los eventos deberán ser enviados mediante el mecanismo Observer y almacenados en MongoDB.

REQ-FN-013 - Exportación de resultados

Requisito:

El sistema **deberá permitir exportar los resultados de una votación en formato JSON o CSV cuando el usuario autenticado posea el permiso `REPORTES\_GENERAR`**.

Formatos permitidos

- JSON.

- CSV.

Excepción

Cuando el formato solicitado no sea soportado, el sistema deberá responder HTTP 400.

La selección del algoritmo de exportación deberá realizarse mediante Strategy Pattern.

---

5. Requisitos no funcionales

REQ-NF-001 - Seguridad de autenticación

El sistema **deberá proteger los endpoints restringidos mediante tokens JWT válidos durante el acceso a recursos protegidos**.

REQ-NF-002 - Autorización

El sistema **deberá verificar los permisos asociados al usuario antes de ejecutar una operación protegida**.

Cuando el usuario esté autenticado pero no posea el permiso requerido, el sistema deberá responder HTTP 403.

REQ-NF-003 - Manejo estandarizado de errores

El sistema **deberá representar los errores HTTP mediante el tipo de contenido `application/problem+json` cuando ocurra una condición de error controlada**.

La respuesta deberá incluir como mínimo:

- `type`

- `title`

- `status`

- `detail`

- `instance`

REQ-NF-004 - Rendimiento de consultas

El sistema **deberá utilizar Redis como caché para disminuir consultas repetitivas de resultados hacia PostgreSQL cuando exista información válida almacenada temporalmente**.

El TTL deberá poder configurarse mediante variable de entorno.

REQ-NF-005 - Disponibilidad ante fallas de Redis

El sistema **deberá continuar proporcionando resultados desde PostgreSQL cuando Redis no se encuentre disponible**.

La indisponibilidad de Redis no deberá impedir el inicio o funcionamiento principal de la API.

REQ-NF-006 - Tolerancia a fallos

El sistema **deberá utilizar Circuit Breaker para reducir llamadas repetitivas hacia Redis cuando se detecten fallos consecutivos**.

El Circuit Breaker deberá manejar los estados:

- CLOSED.

- OPEN.

- HALF_OPEN.

REQ-NF-007 - Integridad de datos

El sistema **deberá utilizar restricciones de PostgreSQL para proteger la integridad de la información transaccional durante las operaciones electorales**.

Se deberán utilizar cuando corresponda:

- Claves primarias.

- Claves foráneas.

- Restricciones UNIQUE.

- Restricciones CHECK.

REQ-NF-008 - Auditabilidad

El sistema **deberá conservar eventos relevantes de operación en MongoDB sin acoplar directamente los servicios de negocio al almacenamiento de auditoría**.

REQ-NF-009 - Mantenibilidad

El sistema **deberá organizar el código por responsabilidades mediante rutas, controladores, servicios, repositorios, middlewares, gateways, eventos y estrategias para facilitar su modificación y prueba**.

REQ-NF-010 - Interoperabilidad

El sistema **deberá exponer sus funcionalidades mediante una API REST documentada utilizando OpenAPI 3.0.3 para permitir su consumo por clientes compatibles con HTTP**.

REQ-NF-011 - Configurabilidad

El sistema **deberá obtener las configuraciones dependientes del entorno mediante variables de entorno sin almacenar credenciales productivas directamente en el código fuente**.

REQ-NF-012 - Capacidad de prueba

El sistema **deberá permitir la ejecución de pruebas automatizadas unitarias y de integración para verificar reglas de negocio, seguridad y patrones críticos**.

---

6. Interfaces externas

6.1 API REST

Prefijo:

`/api/v1`

Recursos principales:

- `/auth`

- `/votaciones`

- `/candidatos`

- `/mesas`

- `/resultados`

El contrato completo se encuentra en:

`docs/api/openapi.yaml`

6.2 PostgreSQL

Responsable de la persistencia transaccional.

Esquemas:

- `auth`

- `election`

6.3 Redis

Responsable de la caché temporal de resultados.

Clave general:

`votagt:resultados:votacion:{votacionId}`

6.4 MongoDB

Responsable de la persistencia de auditoría.

Base:

`votagt\_audit`

Colección:

`auditoria`

6.5 Neo4j

Responsable de representar las relaciones de autorización.

Modelo:

`Usuario -> TIENE\_ROL -> Rol -> OTORGA -> Permiso`

---

7. Reglas de negocio

RN-01: Una votación deberá poseer una fecha de finalización posterior a su fecha de inicio.

RN-02: Un número de lista no deberá repetirse entre candidatos pertenecientes a la misma votación.

RN-03: El código de una mesa no deberá repetirse dentro de la misma votación.

RN-04: Los votos deberán ser números enteros mayores o iguales a cero.

RN-05: Un candidato y una mesa utilizados en un resultado deberán pertenecer a la misma votación.

RN-06: No deberá existir más de un resultado para la misma combinación de mesa y candidato.

RN-07: Las operaciones protegidas deberán comprobar autenticación y autorización.

RN-08: El registro de un nuevo resultado deberá invalidar la caché correspondiente a la votación.

RN-09: PostgreSQL deberá permanecer como fuente principal de los datos electorales.

RN-10: Redis no deberá considerarse almacenamiento permanente.

---

8. Trazabilidad

Los requisitos definidos en este documento deberán estar relacionados con:

- Historias de usuario.

- Escenarios BDD.

- Endpoints OpenAPI.

- Casos de prueba Postman.

- Componentes backend.

- Pruebas automatizadas.

La matriz detallada se encuentra en:

`docs/agile/matriz\_trazabilidad.md`

---

9. Requisitos fuera del alcance implementado actual

El Product Backlog contempla funcionalidades adicionales relacionadas con administración completa de usuarios y mantenimiento avanzado de roles.

Estas funcionalidades podrán formar parte de posteriores incrementos del sistema y no deben presentarse como endpoints terminados mientras no exista implementación ejecutable y evidencia de prueba.

---

10. Criterios generales de aceptación

El sistema será considerado conforme con esta especificación cuando:

- Los endpoints implementados respondan según el contrato OpenAPI.

- Las operaciones protegidas requieran autenticación y permisos.

- PostgreSQL mantenga la información transaccional.

- Redis implemente Cache-Aside.

- El sistema continúe funcionando cuando Redis esté temporalmente fuera de servicio.

- Los eventos de auditoría sean registrados en MongoDB.

- Las relaciones de autorización puedan representarse en Neo4j.

- Los escenarios BDD estén relacionados con los requisitos.

- Las pruebas automatizadas finalicen sin fallos.

- La colección Postman permita reproducir las operaciones principales.

---

11. Referencias del proyecto

- Contrato API: `docs/api/openapi.yaml`

- BDD: `features/`

- Trazabilidad: `docs/agile/matriz\_trazabilidad.md`

- Persistencia: `docs/database/`

- ADR: `docs/architecture/`

- Código backend: `src/`

- Pruebas: `tests/`